

TABLE OF CONTENTS

SL. NO.	TOPIC
1	ABSTRACT
2	PROBLEM DEFINATION AND OBJECTIVES
3	LITERATURE REVIEW
4	DESIGN
5	REQUIREMENTS ANALYSIS AND FEASIBILITY
6	CODE/MODULE/WORKING PROTOTYPE DEVELOPED
7	SYSTEM DESIGN
8	PROPOSED VIABLE SOLUTION
9	CONCLUSION
10	REFERENCES

1. ABSTRACT

Selecting music that aligns with a user's emotional state is often a challenge due to the overwhelming volume of songs available on digital streaming platforms. Traditional recommendation systems typically rely on historical listening data or genre clustering, which may not accurately reflect a user's real-time mood.

The proposed **Moodify** is a web-based application designed to bridge this gap by recommending songs based on user-selected mood categories such as Happy, Sad, Energetic, or Relaxed. The system is built using a **modular full-stack architecture**, utilizing **React.js** for a responsive frontend and **Firestore** for backend services, including Authentication and Firestore Database.

Key features include secure user authentication, real-time mood-based song filtering, and an integrated music player that supports full-track playback via external APIs (such as YouTube or Spotify). The application also maintains user-specific data, such as "Favorites" and "Listening History," stored securely in the cloud. By leveraging modern web technologies and a serverless architecture, the system ensures scalability, high availability, and a personalized listening environment without the need for premium subscriptions.

2. PROBLEM DEFINITION & OBJECTIVES

2.1 PROBLEM DEFINITION

In the current digital music landscape, users are presented with millions of tracks. While this abundance offers variety, it often leads to "choice paralysis," where users struggle to find music that suits their immediate emotional context.

- **Lack of Emotional Context:** Existing platforms like Spotify or Apple Music heavily rely on collaborative filtering (what others liked) or content-based filtering (past history). They often fail to capture the user's *current* state of mind.
- **Playback Restrictions:** Many free-tier versions of popular streaming services restrict users to 30-second previews or shuffle-only play, hindering the listening experience.
- **Accessibility:** Complex AI-based mood systems often require high computational power or expensive hardware, making them inaccessible for simple web-based use cases.

Therefore, there is a distinct need for a lightweight, mood-aware music recommendation system that offers free, full-length playback and immediate personalization.

2.2 OBJECTIVES

The primary objectives of this project are:

1. **To implement secure user authentication** using Firebase Auth to manage user sessions and profiles.
2. **To develop a recommendation engine** that fetches and filters songs based on mood tags using external Music APIs.
3. **To integrate an embedded music player** that allows for legal, full-length playback of recommended tracks within the application.
4. **To create a persistent storage system** using Cloud Firestore to save user preferences, favorite songs, and listening history.

3. LITERATURE REVIEW

3.1 EXISTING SYSTEMS:

Current market leaders in music streaming include:

- Spotify: Uses sophisticated AI for "Daily Mixes" and "Discover Weekly." However, these are largely based on long-term listening habits rather than immediate mood input.
- YouTube Music: Offers excellent video integration but often mixes music with non-musical content, and background play is locked behind a paywall.
- Wynk / Gaana: Popular in regional markets but often cluttered with ads and lacking in precise mood-based filtering for free users.

3.2 RECENT RESEARCH & STUDIES:

A review of recent academic papers reveals various approaches to music recommendation:

- Emotion-Based Music Recommendation Using Sentiment Analysis (IEEE, 2020): This study explored using Natural Language Processing (NLP) to analyze user text (e.g., social media posts) to determine mood [1]. While effective, it requires users to type out their feelings, which can be cumbersome.
- Affective Music Recommendation Using Deep Learning (Elsevier, 2021): Researchers used Deep Neural Networks (DNN) to map song audio features (tempo, pitch) to emotion tags [2]. This provides high accuracy but requires significant backend processing power.
- Context-Aware Systems (ACM, 2022): This survey highlighted the importance of "context" (time, location, weather) in music selection [3]. It concluded that explicit user input (selecting a mood) remains one of the most reliable methods for short-term personalization.
- Facial Detection Based Emotion Analysis (IEEE, 2024): Recent developments have introduced systems utilizing Convolutional Neural Networks (CNN) to detect emotions from live video feeds [4]. These systems capture real-time facial expressions to personalize music choices, though they require camera permissions and higher computational resources on the client side.
- Hybrid Recommendation Approaches (IEEE, 2023): To overcome the limitations of single-method systems, hybrid approaches combining content-based and collaborative filtering have been proposed [5]. These studies demonstrate that integrating user profile

data with mood-based filtering significantly improves user satisfaction and solves the "cold start" problem.

Table 3.1. Comparison table

Feature	Traditional Systems	Proposed System
Basis of Recommendation	User history / similarity	Mood selection or automatic detection
Playback	Premium-only full playback	Free YouTube embedded playback
Emotion Detection	Limited	Optional sentiment analysis
Personalization	Based on usage	Based on current emotional state

3.3 ADVANTAGES OF EXISTING LITERATURE:

- Provides strong foundations in AI-based music tagging and mood modeling.
- Proven that emotionally aligned music improves engagement and user satisfaction.
- Sentiment and emotion recognition models are mature, enabling integration.
- APIs like Spotify and YouTube facilitate scalable implementation.

3.4 LIMITATIONS:

- Full song playback is often restricted to premium users.
- Sentiment-based models require large datasets for accuracy.
- AI systems may be resource-intensive and less beginner-friendly.
- Contextual factors (weather, location) are often ignored.

The literature shows that mood-aware music recommendation is feasible and highly beneficial for personalized listening. Combining APIs with emotion detection provides both accessibility and functional depth suitable for a mini-project.

4. DESIGN

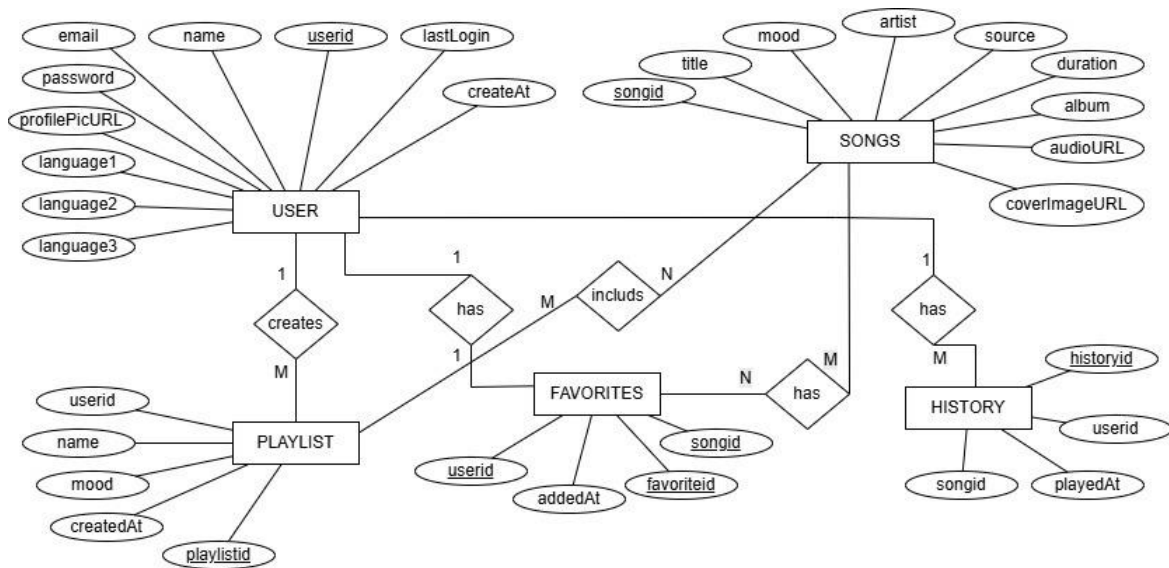


Figure 4.1. ER Diagram of Moodify

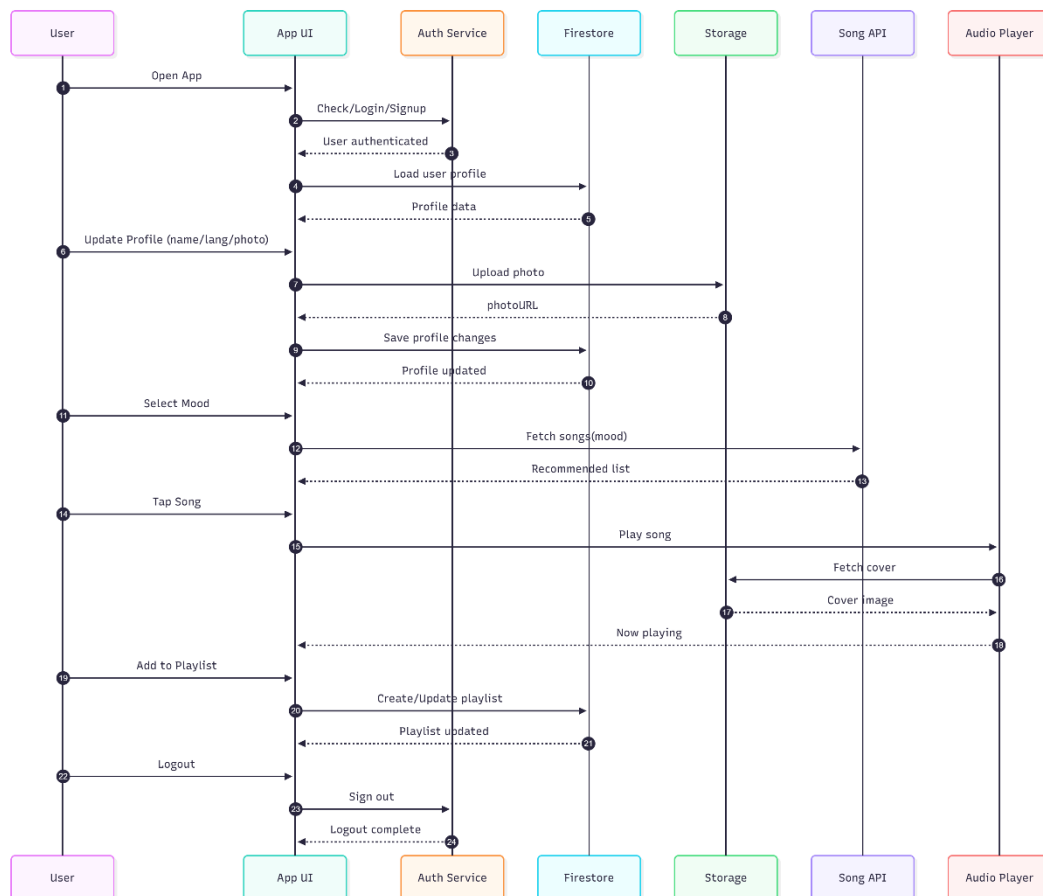


Figure 4.2. Sequence Diagram of Moodify

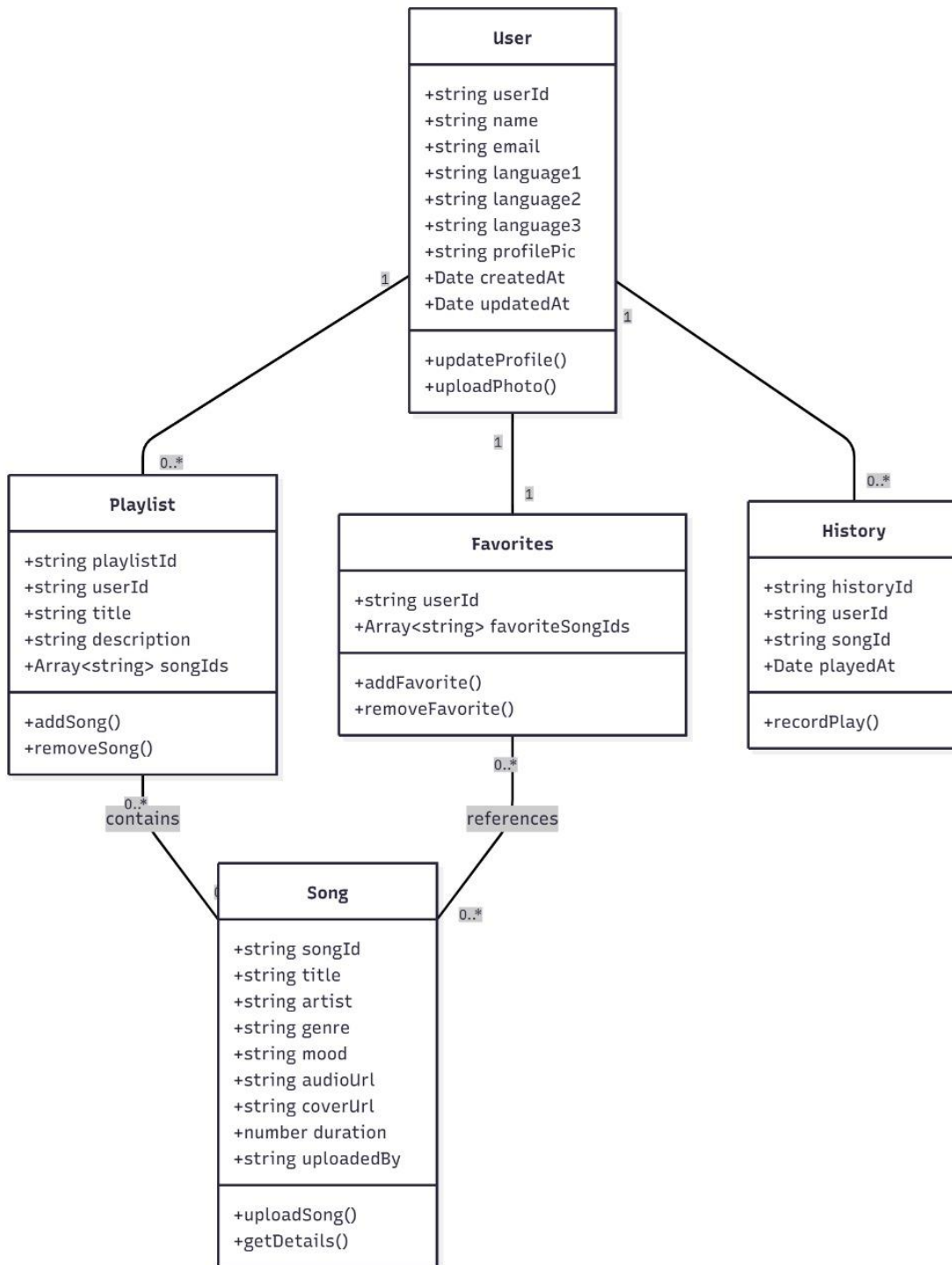


Figure 4.3. Class Diagram of Moodify

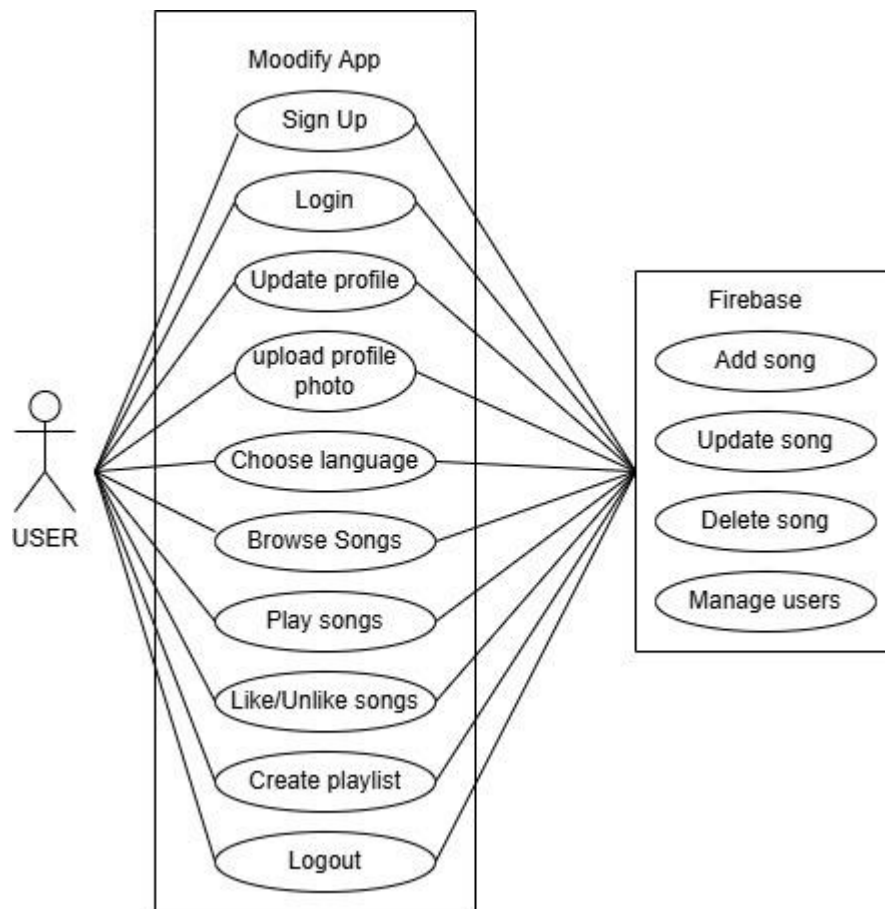


Figure 4.4. Use Case Diagram of Moodify

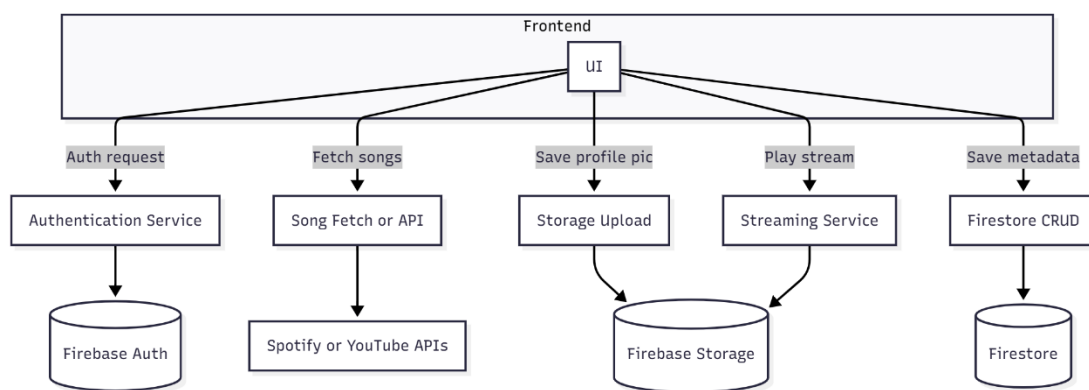


Figure 4.5. Work Flow Diagram of Moodify

5. REQUIREMENTS ANALYSIS & FEASIBILITY

5.1 FEASIBILITY STUDY

5.1.1 Technical Feasibility: The project uses widely supported technologies. React.js is the industry standard for UIs, and Firebase provides a reliable backend-as-a-service. The availability of public APIs (YouTube Data API, Spotify Web API) ensures data availability.

5.1.2 Economic Feasibility: The project is highly cost-effective. Firebase offers a generous "Spark" free tier, and the development tools (VS Code, Git) are open source. No licensing fees are required.

5.1.3 Operational Feasibility: The web-based nature of the app means it requires no installation. It is accessible on any device with a browser, ensuring high usability.

6. FUNCTIONAL/NON-FUNCTIONAL REQUIREMENTS

6.1 FUNCTIONAL REQUIREMENTS

1. **Registration/Login:** Users must be able to create an account and log in securely.
2. **Mood Filtering:** The system must accept a mood input and return a curated list of songs.
3. **Media Playback:** The system must play audio/video without redirecting the user to another site.
4. **Profile Management:** Users should be able to update their profile picture and view their account details.
5. **Data Persistence:** The " Favorites " list must be saved and retrieved across different login sessions.

6.2 NON-FUNCTIONAL REQUIREMENTS

1. **Responsiveness:** The UI must adapt to different screen sizes (Mobile, Tablet, Desktop).
2. **Performance:** API responses should be processed and displayed within 2-3 seconds.
3. **Scalability:** The backend should handle multiple concurrent users without crashing (managed by Firebase).
4. **Security:** User data must be protected using authentication tokens and database security rules.

7. SYSTEM DESIGN

The Moodify system follows a Modular Full-Stack Architecture, ensuring separation of concerns and maintainability.

7.1 ARCHITECTURAL LAYERS

7.1.1 Frontend Layer (User Interface):

Built with React.js, this layer manages the visual elements. It includes components for the Navigation Bar, Mood Cards, Player Controls, and Profile Settings. It communicates with the backend via asynchronous HTTP requests.

7.1.2 API Layer (Communication):

This layer acts as the bridge between the frontend and external data sources. It formats requests (e.g., GET /search?q=happy+music) and parses the JSON responses from the YouTube/Spotify APIs.

7.1.3 Backend Layer (Business Logic):

Powered by Node.js (via Firebase Cloud Functions or direct SDK integration). It handles logic such as mapping specific moods to genre keywords (e.g., "Sad" → "Acoustic", "Blues").

7.1.4 Database Layer (Data Storage):

Cloud Firestore is used as the NoSQL database. It stores data in documents and collections, allowing for flexible schema updates as the application evolves.

7.1.5 Deployment Layer:

The application is hosted on Firebase Hosting, which provides a global Content Delivery Network (CDN) for fast loading speeds.

7.2 SYSTEM WORKFLOW

1. **Input:** User selects a mood (e.g., "Energetic").
2. **Mapping:** The backend maps "Energetic" to genres like "Pop," "Rock," or "EDM."
3. **Fetch:** The system queries the Music API for top tracks in these genres.
4. **Render:** The results are displayed as a list of playable cards.
5. **Interaction:** The user plays a song; the embedded player loads the stream.
6. **Save:** If the user clicks "Favorite," the song ID is pushed to the user's document in Firestore.

8. PROPOSED VIABLE SOLUTION

8.1 VIABILITY JUSTIFICATION

The proposed solution is considered viable for the following reasons:

1. **Cost-Effectiveness:** By utilizing the Free Tier of Firebase and public APIs, the operational cost is near zero, making it sustainable for a student project or startup prototype.
2. **Legal Compliance:** Unlike systems that download MP3s illegally, this solution uses Embedded Players (e.g., YouTube IFrame API). This ensures that artists and platforms are respected, and the app complies with copyright laws.
3. **Scalable Technology:** React and Firebase are industry-standard tools. The knowledge gained in building this system is directly applicable to professional software development roles.
4. **User-Centric Design:** The solution directly addresses the "Choice Paralysis" problem by simplifying the decision-making process down to four distinct emotional choices.

9. CONCLUSION

The Mood-Based Music Recommendation Web App successfully demonstrates the potential of integrating web technologies with human emotion. By shifting the focus from "what did I listen to yesterday?" to "how do I feel right now?", the application provides a more relevant and engaging user experience.

The implementation of React.js ensured a smooth, dynamic interface, while Firebase provided a robust, serverless backend that handled authentication and data storage securely. The integration of external APIs allowed for a vast library of music without the need for massive local storage.

Future Scope: Moving forward, the project can be enhanced by:

- **AI Integration:** Implementing a facial recognition module using Python and OpenCV to automatically detect mood via the webcam.
- **Mobile App:** converting the codebase to React Native for iOS and Android support.
- **Social Features:** Allowing users to share their "Mood Playlists" with friends.

10. REFERENCES

- [1] M. Y. Chen et al., "Emotion-based music recommendation using sentiment analysis," *IEEE Trans. Affective Comput.*, vol. 11, no. 2, pp. 345–354, 2020.
- [2] M. A. Talaghat, E. Parvinnia, and R. Boostani, "Content-based music recommender system based on music emotion using deep learning," *Elsevier Digital Signal Processing*, vol. 11, 2021.
- [3] K. Lee and J. Park, "Context-aware music recommendation systems: A survey," *ACM Comput. Surv.*, vol. 54, no. 6, 2022. A. Sharma, R. Gupta, and S. Kumar, "Music Recommendation System Using Facial Detection Based Emotion Analysis," in *2024 IEEE International Conference on Computing, Communication and Security (ICCCS)*, pp. 1-6, 2024.
- [4] S. Gupta and R. Kumar, "Personalized Mood-Based Song Recommendation System Using a Hybrid Approach," in *2023 IEEE International Conference on Research Methodologies in Knowledge Management (RMKMATE)*, pp. 1-5, 2023.
- [5] Firebase Documentation, "Build and run your app with Firebase," Google Developers. [Online]. Available: <https://firebase.google.com/docs>.
- [6] React Documentation, "Getting Started with React," Meta Open Source. [Online]. Available: <https://react.dev>.
- [7] Google Developers, "YouTube Data API Overview," [Online]. Available: <https://developers.google.com/youtube/v3>.